Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	12 October 2026
Team ID	SWUID2026403439
Project Name	TixBox – Online Movie, Event & Entertainment Ticket Booking System
Maximum Marks	4 Marks

Technical Architecture:

TixBox is designed as a scalable, self-contained 3-tier web system: a React single-page application (presentation layer), a stateless Spring Boot REST API (business logic) and a relational database with an optional Redis cache (data layer). The client talks to the API over HTTPS using JSON; Spring Security with a JWT filter authenticates every request and enforces role-based access. External integrations are limited to an SMTP email server and a mock payment gateway, keeping the platform secure, testable and easy to extend with a real gateway (Razorpay/Stripe) later.

Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	Responsive single-page web app (320 px and above) for guests, customers, cinema managers, organizers and admins: city selection, search/filters, interactive seat map with countdown timer, booking and My Bookings	HTML5, CSS3, JavaScript / React.js, Bootstrap / Tailwind CSS
2.	Application Logic-1	Authentication and RBAC (JWT, BCrypt, OTP verification), profiles, location, movie/cinema/screen/show management, search, filter and sort	Java 17, Spring Boot 3.x, Spring Security, Spring Data JPA
3.	Application Logic-2	Booking engine: seat selection and locking with 10-min expiry, F&B, coupons, order summary, cancellation and refunds, events and ticket categories	Spring Boot services, Spring @Scheduled jobs (lock expiry, reconciliation)
4.	Application Logic-3	Mock payment gateway (initiate, verify, refund with HMAC-signed callbacks), QR e-ticket generation and staff verification, reviews, notifications, admin/organizer analytics	Spring Boot, ZXing (QR), HMAC / JWT-signed tokens, Chart.js / Recharts
5.	Database	Stores users, roles, cities, movies, cinemas, screens, shows, show seats, bookings, payments, tickets, coupons, events, reviews and notifications	PostgreSQL 14+ (or MySQL 8+), Flyway migrations
6.	Cache / Seat-Lock Store	Fast seat locks (SET NX with TTL), cached movie lists and rate limiting	Redis 7+ (optional)
7.	File Storage	Movie posters, banners, event images and profile pictures (JPG/PNG/WebP, max 2 MB)	Local / cloud object storage (e.g. Cloudinary, AWS S3)

S.No	Component	Description	Technology
8.	Email / Notifications	Verification, booking confirmation, cancellation and refund emails plus in-app notifications, sent asynchronously	SMTP (Gmail / Mailtrap), Spring Mail, @Async
9.	External API	Simulated payment service for UPI, cards, net banking and wallets; no real money or card data handled	Mock Payment Gateway module (REST)
10.	Infrastructure	Static frontend build served by Nginx, backend as an executable JAR in Docker, TLS termination and /api proxy	Docker, Docker Compose, Nginx, HTTPS (TLS 1.2+)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
11.	Open-Source Frameworks	Frontend, backend, ORM and security frameworks	React.js, Node.js 18+ (build), Bootstrap, Tailwind CSS, Spring Boot, Hibernate
12.	Security Implementations	Stateless JWT with refresh tokens, RBAC, BCrypt (strength 10+), account lockout after 5 failed logins, rate limiting, CORS allow-list, input validation, signed QR tokens, no card data stored	Spring Security, JWT (RFC 7519), HTTPS/HSTS, OWASP Top 10 / ASVS
13.	Scalable Architecture	3-tier layered architecture with a stateless REST API; seat locks in a shared store so multiple instances can run	MVC, REST/JSON, Redis
14.	Availability / Performance	Seat availability refreshed every 5 s (WebSocket/SSE optional); paginated lists (20 per page); atomic seat locking via DB transactions (SELECT … FOR UPDATE) with optimistic version check	Polling / WebSocket, JPA locking
15.	API Documentation	Versioned REST APIs documented and testable	OpenAPI / Swagger, Postman
16.	Testing	Unit, integration, UI/E2E, performance and security testing (≥70% unit coverage)	JUnit 5, Mockito, Cypress / Selenium, JMeter / k6, OWASP ZAP
17.	Version Control & Tools	Source management, build and development tooling	Git / GitHub, Maven, IntelliJ IDEA, VS Code

References:

React.js Documentation

Spring Boot Documentation

OWASP Top 10

How to draw useful technical architecture diagrams